ECG Arrhythmia Classification on a Microcontroller via Time-Series-to-Image Encoding and the Z-Ant Inference Framework

Rocco Panizzi[1], Zeno Pellizzari[1], and Michele Polise[1]

[1]Politecnico di Milano, Corso di Laurea in Ingegneria Informatica

Abstract — This work presents an end-to-end pipeline for time-series classification on a resource-constrained embedded platform, developed as a computer engineering project in collaboration with Z-Ant, an open-source neural network inference engine. The goal is to enable Z-Ant's inference engine — which operates natively on tensor-shaped images — to accept raw time series as input, by encoding them as images through Gramian Angular Fields and Markov Transition Fields prior to inference.

The chosen application domain is cardiac arrhythmia detection using the MIT-BIH Arrhythmia dataset. Each heartbeat, originally represented as 187 amplitude samples, is resampled to 64 points to fit within the memory constraints of the target hardware (Arduino Nicla Vision, Cortex-M7), then encoded into a three-channel 64×64 image combining GASF, GADF, and MTF. A lightweight convolutional network (≈61 000 parameters) achieves 97 % accuracy and a macro-average F1 score of 0.88 on the held-out test set. On-board validation confirmed end-to-end functionality: inference on a single ECG window takes approximately 2 seconds on the Nicla Vision.

1. INTRODUCTION

Time-series classification is a fundamental problem in many engineering and medical domains. A particularly relevant instance is the automatic detection of cardiac arrhythmias from electrocardiogram signals: reliably identifying anomalous heartbeat patterns in real time has direct clinical value, and the availability of established benchmarks, such as the MIT-BIH Arrhythmia dataset, makes it a well-suited testbed for new classification approaches.

Convolutional neural networks have demonstrated strong performance in image classification. However, their direct application to raw one-dimensional time series is non-trivial: the local spatial structure that convolutional filters exploit is not naturally present in a 1D signal. Gramian Angular Fields (GASF and GADF) address this by mapping a normalized time series to a square matrix, where each entry encodes the angular relationship between a pair of time points, effectively recasting temporal structure as spatial structure that a CNN can process. The Markov Transition Field (MTF) complements this representation by encoding, for each pair of time indices, the empirical probability of transitioning between the amplitude states occupied at those instants, capturing the signal's dynamic behavior rather than its instantaneous geometry. Stacking GASF, GADF, and MTF as three channels of a single $N \times N$ image produces a compact, information-rich representation that a standard CNN architecture can classify directly.

This work integrates this encoding pipeline into Z-Ant [6], an open-source neural network inference engine for embedded systems that compiles ONNX models into self-contained Zig libraries deployable on microcontrollers. While Z-Ant already supported convolutional inference on image data, this project extends it to accept raw time series as input; concretely, the GASF, GADF, and MTF encoding modules, the compound channel assembler, and a C-callable endpoint were designed and implemented from scratch in Zig. The correctness of each module was validated by comparing its numerical output with a reference Python implementation on identical inputs, ensuring pixel-level agreement before any hardware test was performed.

The target deployment platform is the Arduino Nicla Vision, a microcontroller board on which end-to-end validation was performed by streaming serialized ECG windows from a host PC over a serial connection and reading back the predicted class and raw confidence scores over the same link.

2. IMAGING TIME SERIES

2.1 Gramian Angular Fields (GASF and GADF)

Gramian Angular Fields encode a time series as a square matrix whose entry (i, j) captures the angular relationship between time points i and j, producing a structured 2-D representation that preserves temporal correlations and is directly consumable by a convolutional network. Two complementary variants exist: the Gramian Angular *Summation* Field (GASF), based on the cosine of the sum of angles, and the Gramian Angular *Difference* Field (GADF), based on the sine of the difference.

Polar encoding. Given a time series $X = (x_1, \ldots, x_N)$, values are rescaled to $[-1, 1]$:

$$\tilde{x}_i = \frac{(x_i - \max X) + (x_i - \min X)}{\max X - \min X}. \tag{1}$$

Each normalized sample is then interpreted as the cosine of a polar angle

$$\phi_i = \arccos(\tilde{x}_i), \qquad \phi_i \in [0, \pi], \tag{2}$$

with per-sample quantities $c_i = \tilde{x}_i$ and $s_i = \sqrt{1 - \tilde{x}_i^2}$ computed once in $O(N)$.

GASF encodes the pairwise cosine of the *sum* of angles:

$$G_{ij}^{\text{GASF}} = \cos(\phi_i + \phi_j) = c_i c_j - s_i s_j. \tag{3}$$

In matrix form:

$$\text{GASF} = \mathbf{c}^\top \mathbf{c} - \mathbf{s}^\top \mathbf{s}. \tag{4}$$

GASF is *symmetric* ($G_{ij} = G_{ji}$) and bounded in $[-1, 1]$. The main diagonal encodes the self-correlation of each sample: $G_{ii} = \cos(2\phi_i) = 2\tilde{x}_i^2 - 1$.

Gramian Angular Difference Field. GADF encodes the pairwise sine of the *difference* of angles:

$$G_{ij}^{\text{GADF}} = \sin(\phi_i - \phi_j) = s_i c_j - c_i s_j. \tag{5}$$

In matrix form:

$$\text{GADF} = \mathbf{s}^\top \mathbf{c} - \mathbf{c}^\top \mathbf{s}. \tag{6}$$

GADF is *anti-symmetric* ($G_{ij} = -G_{ji}$), bounded in $[-1, 1]$, and has a zero main diagonal since $\sin(0) = 0$.

Implementation. Both Zig implementations (`lean_gasf` and `lean_gadf`) follow the algebraic forms in (3) and (5) directly. Given the normalised input $\tilde{x}$, each function first fills the sine buffer $s_i = \sqrt{1 - \tilde{x}_i^2}$ in $O(N)$, then fills the pre-allocated $N \times N$ output matrix (row-major) in $O(N^2)$. No dynamic allocation occurs; all buffers are owned by the caller, keeping both routines suitable for memory-constrained embedded targets such as the Arduino Nicla Vision.

Complementarity with MTF. GASF captures *static* angular similarity between time instants, while GADF captures *directional* change. Wang and Oates [5] observe that the two representations act as orthogonal channels: combining them with the MTF into a three-channel RGB-like image allows the CNN to exploit both static amplitude correlations and dynamic transition information simultaneously.

2.2 Markov Transition Field (MTF)

The Markov Transition Field (MTF) [5] explicitly represents the signal's dynamic behavior by encoding empirical transition probabilities between discretized states, producing a global picture of the signal dynamics that is independent of the angular geometry used by the Gramian fields.

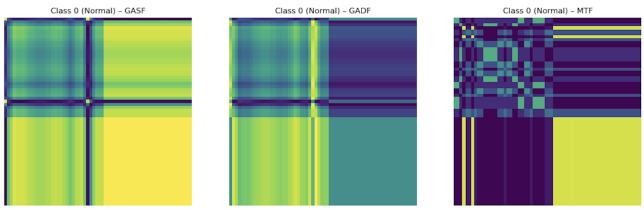

Figure 1: Three-channel encoding: GASF (R), GADF (G), MTF (B).

Quantile Binning

Given a time series $X = (x_1, \ldots, x_N)$, the value range is partitioned into Q bins via equal-count (quantile) boundaries. The $Q-1$ interior boundaries are placed at positions

$$e_k = X_{\text{sorted}}\left[\left\lfloor \frac{kN}{Q} \right\rfloor\right], \qquad k = 1, \ldots, Q-1, \tag{7}$$

where X_{sorted} denotes the sorted copy of X. Each sample is then assigned a bin index

$$q_i = \sum_{k=1}^{Q-1} \mathbf{1}[x_i \geq e_k] + 1 \;\in\; \{1, \ldots, Q\}, \tag{8}$$

so that approximately $\lfloor N/Q \rfloor$ samples fall into each bin. Using equal-count rather than equal-width boundaries ensures that every state is represented by a comparable number of observations, preventing sparsity in the transition matrix.

Markov Transition Matrix

Let $W \in \mathbb{R}^{Q \times Q}$ be the matrix of observed one-step transition counts:

$$W_{kl} = \sum_{t=1}^{N-1} \mathbf{1}[q_t = k]\, \mathbf{1}[q_{t+1} = l], \qquad k, l \in \{1, \ldots, Q\}. \tag{9}$$

We then row-normalize and get the row-stochastic matrix $\widehat{W}$:

$$\widehat{W}_{kl} = \frac{W_{kl}}{\sum_{l'=1}^{Q} W_{kl'}}, \tag{10}$$

with the convention that any row whose sum is zero is left as the zero vector. The entry $\widehat{W}_{kl}$ represents the empirical probability of transitioning from bin k to bin l in a single time step.

MTF Construction

The $N \times N$ Markov Transition Field matrix M is defined by

$$M_{ij} = \widehat{W}_{q_i,\, q_j}, \qquad i, j \in \{1, \ldots, N\}. \tag{11}$$

The entry M_{ij} reports the transition probability between the quantile state occupied at time i and the state occupied at time j, extending the one-step Markov matrix to arbitrary — including non-consecutive — pairs of time indices.

3. Z-ANT ENDPOINT

The Z-Ant endpoint constitutes the integration layer between the mathematical pipeline described in the previous sections and the embedded inference engine running on the target hardware. Its role is to expose a single, self-contained function that accepts a raw time series and produces a tensor ready for consumption by the convolutional network with no dynamic memory allocation when compiled with the -Dstatic flag, and no runtime dependencies beyond the compiled library itself.

3.1 The `compound` Module

Before the endpoint can be defined, the three encoding channels must be combined into a single tensor. The `compound.zig` module handles this, merging GASF, GADF, and MTF into a flat CHW buffer of shape $[3, N, N]$ suitable for a CNN that expects an RGB-like input.

The output layout is organized as three consecutive channels of $N \times N$ values:

- **Channel 0 — GASF.** Values originally in $[-1, 1]$ are rescaled to $[0, 1]$ via the affine map $x \mapsto (x+1)/2$.
- **Channel 1 — GADF.** Same rescaling as GASF.
- **Channel 2 — MTF.** Values are already in $[0, 1]$ by construction and are copied as-is.

Normalization of the input series is performed once inside `lean_compound`, and the resulting normalized series is shared by all three encoding stages, avoiding redundant computation and ensuring that GASF, GADF, and MTF operate on identical preprocessed inputs.
The module exposes two levels of API:

- `lean_compound(input, norm, q, *scratch, output)`. All intermediate buffers (`norm_buf`, `cosines_buf`, `sines_buf`, `sorted_buf`, `bins_buf`, `matrix_buf`) are owned by a `CompoundScratch` struct that is allocated once and reused across many series.
- `batchToRGBImageF32(allocator, inputs, norm, q)` — a higher-level wrapper that processes a batch of B series of length N and returns a flat NCHW buffer of shape $[B, 3, N, N]$. The scratch is allocated once outside the loop and reused for each element of the batch.

3.2 The C-Callable Endpoint

The file `timeseries_to_image_api.zig` exposes the following C-callable function:

```
pub export fn zant_timeseries_to_image(
    signal: [*]const f32,
    output: [*]f32,
) void
```

The two parameters $N = 64$ and $Q = 4$ are compiled as constants (this choice is explained afterward). All intermediate buffers are declared as static module-level arrays:

```
var s_norm:   [N]f32   = undefined;
var s_cos:    [N]f32   = undefined;
var s_sin:    [N]f32   = undefined;
var s_sorted: [N]f32   = undefined;
var s_bins:   [N]usize = undefined;
var s_matrix: [Q*Q]f32 = undefined;
```

At runtime, the function constructs a `CompoundScratch` pointing to these static buffers, calls `lean_compound`, and clamps the output to $[0, 1]$. The result is a complete $3 \cdot 64 \cdot 64 = 12{,}288$-element CHW tensor written directly into the caller-supplied `output` pointer. No heap allocation takes place at any point during execution.

Design Rationale

The choices made in the endpoint design are dictated by the constraints of the target platform (Arduino Nicla Vision, Cortex-M7):

- **Flat C signature.** A two-pointer interface maximizes compatibility with any Arduino or C++ toolchain and requires no struct marshalling across the FFI boundary.
- **Compile-time constants for N and Q.** Hard-coding the shape parameters allows the compiler to size all buffers as fixed-length arrays known at compile time, eliminating any runtime shape arithmetic. Changing the model requires recompiling the library, not modifying the firmware.
- **Static module-level buffers.** Placing all working memory in the `.bss` segment makes memory usage fully predictable and rules out out-of-memory conditions by construction — a critical property on microcontrollers where heap fragmentation is unacceptable.
- **Output clamped to $[0, 1]$.** The CNN expects inputs in this range. The clamp guards against floating-point drift that could push GASF or GADF values slightly outside range before the affine rescaling.

3.3 Integration in the Firmware

The endpoint acts as the interface between two worlds: on one side, a flat buffer of 64 ECG samples in the natural format of a serial acquisition; on the other, a `[1, 3, 64, 64]` tensor in CHW layout with values in $[0, 1]$, which is precisely the input format expected by the CNN.

In the Arduino sketch (`tiny_hack.ino`), the firmware declares the endpoint and the Z-Ant inference function as external C symbols, and allocates all working buffers statically in the `.bss` segment:

```
extern "C" void zant_timeseries_to_image(
    const float *signal, float *output);
#include <lib_zant.h>

#define SIGNAL_LEN 64
#define IMAGE_SIZE 64
#define OUT_LEN    5

static float    timeseriesData[SIGNAL_LEN];
static float    imageData[3 * IMAGE_SIZE * IMAGE_SIZE
    ];
static uint32_t inputShape[4] = {1, 3, IMAGE_SIZE,
    IMAGE_SIZE};
```

`timeseriesData` holds the raw ECG window, `imageData` holds the $3 \times 64 \times 64$ CHW tensor, and `inputShape` is a compile-time constant passed to `predict`.

Setup

The `setup()` function initializes the serial interface and brings up the QSPI flash in memory-mapped (XIP) mode. This process is required because the model weights compiled by Z-ANT are stored in the external 16 MB QSPI flash and accessed as if they were ordinary memory — the Cortex-M7 bus reads them transparently once `qspi_enter_mmap` has been called.

```
void setup() {
    Serial.begin(115200);
    delay(8000);

    if (qspi_init_16mb(&hqspi) != HAL_OK) {
        Serial.println("QSPI init FAIL"); for (;;) {}
    }
    if (enable_quad(&hqspi) != HAL_OK) {
        Serial.println("Enable QE FAIL"); for (;;) {}
    }
    if (qspi_enter_mmap(&hqspi) != HAL_OK) {
        Serial.println("XIP FAIL");       for (;;) {}
    }
    Serial.println("READY");
}
```

If any of the three initialization steps fail, the board immediately halts in an infinite loop and reports the failure over serial. Only after `READY` is printed does the host PC begin sending data.

Acquisition

The `loop()` function accumulates incoming bytes into a static receive buffer. Since the serial interface delivers data byte by byte, the sketch waits until exactly $64 \times 4 = 256$ bytes have arrived before proceeding. A single `memcpy` then reinterprets the raw bytes as an array of 64 `float` values in IEEE 754 little-endian format.

```
static uint8_t recvBuf[SIGNAL_LEN * sizeof(float)];
static int     recvCount = 0;

while (Serial.available() > 0 &&
       recvCount < (int)(SIGNAL_LEN * sizeof(float)))
            {
    recvBuf[recvCount++] = Serial.read();
}

if (recvCount >= (int)(SIGNAL_LEN * sizeof(float))) {
    memcpy(timeseriesData, recvBuf, SIGNAL_LEN *
        sizeof(float));
    recvCount = 0;
```

Pre-processing

Once the window is complete, the endpoint is called with the two flat pointers. Internally, it executes normalization, image encoding, assembles the three channels, and clamps the result to $[0, 1]$ — all without touching the heap.

```
    Serial.println("Data received, processing...");
    Serial.flush();

    zant_timeseries_to_image(timeseriesData,
        imageData);

    Serial.println("Conversion OK");
    Serial.flush();
```

After this call, `imageData` contains the complete $3 \times 64 \times 64 = 12{,}288$-element CHW tensor ready for inference.

Inference and Classification

The tensor is passed directly to the Z-ANT-generated `predict` function. No copy or format conversion is needed: the CHW layout and the $[0, 1]$ value range produced by the endpoint match exactly what the CNN expects. The function writes its output logits to a pointer `out` and returns zero on success.

```
    float *out = nullptr;
    int rc = predict(imageData, inputShape, 4, &out);

    if (rc == 0 && out) {
        int best = 0;
        for (int i = 1; i < OUT_LEN; i++)
            if (out[i] > out[best]) best = i;

        Serial.print("CLASS:"); Serial.print(best);
        Serial.print(" SCORES:");
        for (int i = 0; i < OUT_LEN; i++) {
            Serial.print(out[i], 4);
            if (i < OUT_LEN - 1) Serial.print(",");
        }
        Serial.println();
    } else {
        Serial.println("PREDICT_FAIL");
    }
}
```

The argmax over the five output logits yields the predicted class. Both the winning class index and the full score vector are transmitted over serial, allowing the host PC to log raw confidence values for evaluation without requiring any additional firmware modification.

From the perspective of the sketch, the entire mathematical pipeline is a black box behind two pointers. The `.ino` file is entirely agnostic to the internal code: updating the model requires only recompiling the Z-ANT library, leaving the sketch unchanged.

4. CNN TRAINING

4.1 Architecture

The classifier is a lightweight convolutional neural network designed with two constraints in mind: (i) the input is a three-channel 64×64 image, and (ii) every operator must be supported by Z-ANT for float32 inference and deployment on the Arduino Nicla Vision. The network, referred to as(`EcgClassifier`), is built from Conv + ReLU blocks followed by a global average-pooling layer and a single Gemm layer (Figure 2). Strided convolutions (stride 2) replace pooling layers, halving feature map size at each stage while increasing the channel count from 3 to 64. The global average pool collapses each 4×4 feature map to a scalar, producing a 64-dimensional vector for the linear classifier. The total number of trainable parameters is $\approx$61 000, a footprint small enough for embedded inference.

4.2 Dataset and Class Imbalance

Training and evaluation use the MIT-BIH Arrhythmia dataset [3] in the pre-segmented version released by Kachuee et al. [1]: 87 554 training samples and 21 892 test samples, each consisting of 187 amplitude values labelled as one of five classes: Normal (0), Supraventricular ectopic beat (1), Premature ventricular contraction (2), Fusion beat (3), and Paced beat (4).

The dataset is heavily imbalanced: Normal beats ac-

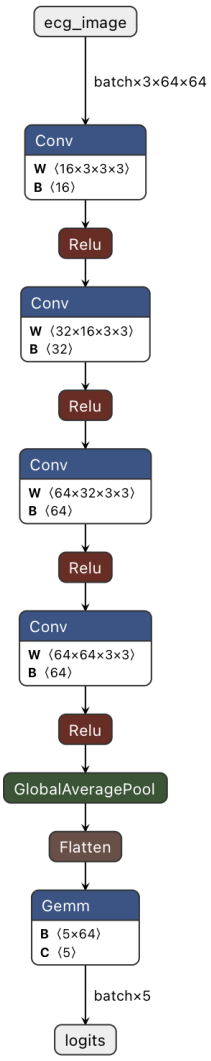

Figure 2: `EcgClassifier` architecture as exported to ONNX.

count for 82.8% of the training samples, while Fusion beats represent only 0.7% (Table 1). A model that predicts class 0 for every input would achieve ≈83% accuracy without learning any discriminative feature. To counter this, each class is weighted in the loss function inversely proportional to its frequency:

$$w_c = \frac{N}{K \cdot n_c}, \tag{12}$$

where N is the total number of training samples, $K = 5$ is the number of classes, and n_c is the number of samples in class c. The resulting weights are $w = [0.006, 0.191, 0.074, 0.663, 0.066]$, forcing the network to pay roughly 110 times more attention to a Fusion beat than to a Normal beat.

4.3 Training Procedure

The model is trained for up to 30 epochs (batch size 64) on an Apple M4 SoC via the MPS backend, using the Adam optimiser [2] with an initial learning rate of 10^{-3}. A `ReduceLROnPlateau` scheduler halves the learning rate whenever the validation loss does not improve for 3 consecutive epochs; early stopping terminates training if the best validation loss is not improved for 5 epochs. Training stopped at epoch 26.

Table 1: MIT-BIH training-set class distribution.

Class	Label	Samples	%
0	Normal	72 471	82.8
1	Supraventricular	2 223	2.5
2	Premature ventricular	5 788	6.6
3	Fusion beat	641	0.7
4	Paced beat	6 431	7.3

Table 2: Per-class metrics on the MIT-BIH test set.

Class	Prec.	Rec.	F1	Supp.
Normal	0.99	0.98	0.99	18 118
Supraventricular	0.70	0.78	0.74	556
Premature ventr.	0.94	0.93	0.94	1 448
Fusion beat	0.69	0.81	0.75	162
Paced beat	0.97	0.98	0.98	1 608
Macro avg	0.86	0.90	0.88	21 892
Weighted avg	0.97	0.97	0.97	21 892

4.4 Results

The model reaches 97 % overall accuracy (Table 2), with weighted-average precision and recall both at 0.97. The weakest classes are Supraventricular (F1 = 0.74) and Fusion beat (F1 = 0.75), which are also the two rarest in the dataset. Importantly, the test set itself mirrors the training-set imbalance, meaning that per-class metrics for minority classes are computed over a small number of samples (556 and 162, respectively) and are therefore less statistically reliable than those for the majority classes. The macro-average F1 of 0.88 provides a more informative measure of balanced classification ability.

5. DEPLOYMENT ON THE NICLA VISION

5.1 On-board pipeline.

Once the 256 bytes are received, the firmware calls `zant_timeseries_to_image`, which executes the full preprocessing chain on the M7 core: min-max normalization, GASF encoding, GADF encoding, MTF encoding with $Q = 4$ quantile bins, and assembly of the three channels into a CHW tensor of shape $[3, 64, 64]$. The tensor is passed directly to `predict`, which runs the quantized CNN and returns five logit scores. The board replies over the serial line with the predicted class index and the full score vector, for example:

```
CLASS:0 SCORES:3.1204,0.2531,0.1003,0.0421,0.0187
```

5.2 Hardware

The target platform is the Arduino Nicla Vision, a compact embedded board built around the STM32H747xx dual-core microcontroller (Cortex-M7 at 480 MHz + Cortex-M4 at 240 MHz)[4], 1 MB of RAM, and 2 MB of internal flash. Inference runs exclusively on the M7 core. The board also mounts 16 MB of external QSPI flash, which is the key

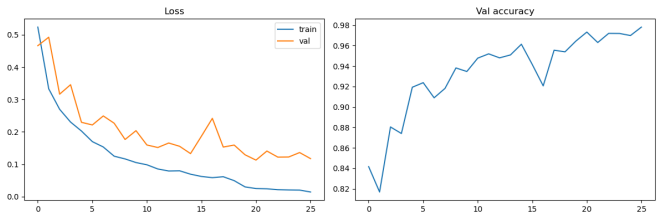

Figure 3: Training and validation loss and accuracy over epochs.

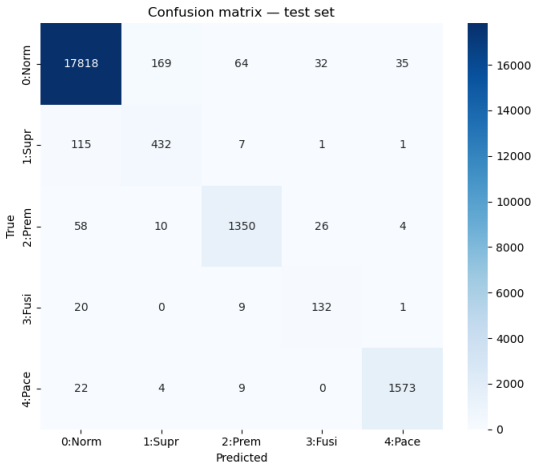

Figure 4: Confusion matrix on the software test set.

resource that makes deployment feasible: the weights of the compiled CNN exceed the internal flash budget and must therefore be stored and read externally using the XIP protocol.

5.3 QSPI XIP

The external flash is accessed in *eXecute In Place* (XIP) mode via the STM32 HAL QSPI driver. Once memory-mapped mode is enabled, the Cortex-M7 bus exposes the QSPI flash as a read-only memory region starting at `0x90000000`: the Z-Ant runtime receives a single pointer, `flash_weights_base`, and reads the model weights directly, with no DMA transfers or intermediate copies.

5.4 Build and flashing

The Zig toolchain produces a static library `libzant.a` targeting `thumb-freestanding / cortex_m7`, with the model weights placed in a dedicated `.flash_weights` linker section. A custom linker script maps that section to the QSPI region at `0x90000000`, keeping it physically separate from the firmware. At flash time, `objcopy` splits the compiled ELF into two binaries — internal firmware and external weights — which are uploaded independently via `dfu-util`: the firmware to internal flash at `0x08040000` and the weights to QSPI at `0x90000000`.

5.5 On-board validation

End-to-end pipeline. The validation setup is deliberately minimal on the host side: the PC is nothing more than a data source and a result collector. All the computational work happens on the microcontroller. The host streams raw ECG windows over a 115200 baud serial link;

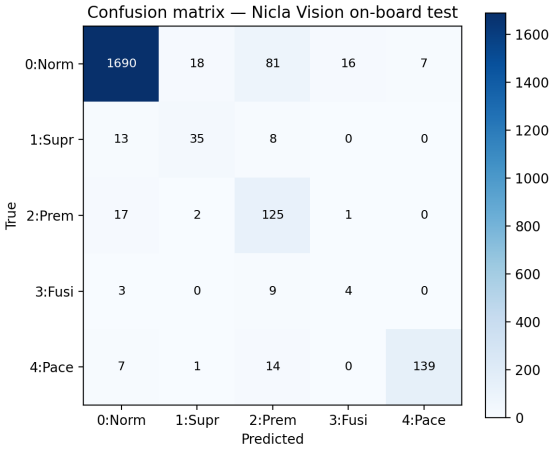

Figure 5: Confusion matrix on the Nicla Vision on-board test set.

the Nicla Vision runs the complete pipeline autonomously and replies with the predicted class and the five softmax scores.

Host-side preprocessing. The only operation performed on the host before transmission is a linear resampling of the raw ECG signal. Each sample in the MIT-BIH dataset consists of 187 amplitude values; the board, however, expects windows of exactly $N = 64$ points, matching the input size the CNN was trained on. The resampling is carried out by `resample_linear`, which interpolates the original 187-point signal onto a uniform 64-point grid:

$$x'_k = x\left(\frac{k \cdot 186}{63}\right), \qquad k = 0, \ldots, 63, \tag{13}$$

where the fractional index is resolved by linear interpolation between the two nearest original samples. The resampled window is then serialized as 64 IEEE 754 single-precision floats (256 bytes) and sent over the serial line.

Results. The on-board test was run on a subset of the MIT-BIH test set comprising 2 190 samples distributed across the five arrhythmia classes. The board achieved an overall accuracy of **91.0%** (1 993 correct out of 2 190), with the confusion matrix reported in Figure 5.

6. EMPIRICAL RESULTS

6.1 Test Set

The on-board evaluation was run on a subset of 2 190 samples drawn from the MIT-BIH test set. The subset mirrors the original class distribution: Normal beats account for 1 812 of the 2 190 samples (82.7 %), while Fusion beats are represented by only 16 samples and Supraventricular beats by 56 (Figure 5). The two evaluation settings are therefore comparable in terms of class distribution, and the macro-average F1 drop observed on-board reflects genuine degradation rather than a distributional artifact.

Table 3: Top-level performance: software vs. on-board.

	Software	**On-board**
Samples	21 892	2 190
Overall accuracy	0.97	0.91
Macro-avg F1	0.88	0.67

6.2 Comparison: Software vs. On-Board

Table 4 shows that the degradation is not uniform. Normal and Paced beats transfer well to the embedded platform (F1 of 0.95 and 0.90, respectively, against 0.99 and 0.98 in software). The degradation is severe for Fusion beats (F1 drops from 0.75 to 0.22) and noticeable for Premature ventricular contractions (0.94 to 0.66) and Supraventricular ectopic beats (0.74 to 0.62).

The per-class breakdown in Table 4 shows that the degradation is not uniform. Normal and Paced beats transfer well to the embedded platform (F1 of 0.95 and 0.90 respectively, against 0.99 and 0.98 in software). The degradation is severe for Fusion beats (F1 drops from 0.75 to 0.22) and noticeable for Premature ventricular contractions (0.94 to 0.66) and Supraventricular ectopic beats (0.74 to 0.62).

Table 4: Per-class metrics: software vs. on-board.

	Software		**On-board**	
Class	Rec.	F1	Rec.	F1
Normal	0.98	0.99	0.933	0.954
Supraventricular	0.78	0.74	0.625	0.619
Premature ventr.	0.93	0.94	0.862	0.655
Fusion beat	0.81	0.75	0.250	0.216
Paced beat	0.98	0.98	0.863	0.902
Macro avg	0.90	0.88	0.707	0.669

Two factors contribute to the observed degradation.

Small support for minority classes. The Fusion beat on-board estimate is based on only 16 samples. At that scale, two or three predictions going in different directions would substantially change the reported F1 of 0.22, making it statistically unreliable. The Supraventricular class, with 56 on-board samples, is only marginally more trustworthy. Any interpretation of per-class results for these two classes should be treated with caution.

Morphological sensitivity to resampling. For some classes, the discriminative features span a narrow amplitude range, which is directly blurred by linear interpolation. The Fusion beat, which by definition blends a normal sinus waveform with a ventricular ectopic pattern, produces a representation that the 64-sample encoding struggles to preserve distinctly enough for reliable classification. The Paced beat, by contrast, is characterized by a sharp pacing spike whose large-amplitude excursion survives resampling well, consistent with its comparatively strong on-board performance ($F1 = 0.90$).

6.3 Computational Cost

Each inference run — serial reception, full signal encoding, and CNN forward pass — takes approximately 2 seconds on the Cortex-M7 at 480 MHz. This is consistent with real-time heartbeat-by-heartbeat classification at resting heart rates ($\lesssim 1\,\text{beat}\,\text{s}^{-1}$), though it would not scale to elevated heart rates without further optimization.

7. PERSPECTIVES AND EXTENSIONS

The results obtained on the Arduino Nicla Vision demonstrate that a full time-series classification pipeline — from raw signal to predicted class — can run entirely on a microcontroller with severe memory and compute constraints. This opens several concrete directions for both future technical development and real-world deployment.

Dataset and training improvements. The two weakest classes in the on-board evaluation are Supraventricular ectopic beat and Fusion beat, which are also the two rarest in the MIT-BIH dataset (2.5% and 0.7% of training samples, respectively). Expanding the training set with a larger, more balanced collection of labeled heartbeats — covering a wider range of patients, recording conditions, and clinical presentations — would directly improve generalization on these minority classes and is the most straightforward path to higher overall accuracy.

Hardware with larger memory. The resampling from 187 to 64 samples is not an algorithmic choice but a hardware constraint: the Nicla Vision's memory budget forces a reduction in signal length, inevitably discarding information. On a device with more available RAM — even a modestly more capable microcontroller — it would be possible to work directly on the original 187-sample windows, or to find an intermediate length that balances memory usage and classification performance. Since the accuracy gap between the software evaluation (97%) and the on-board test (91%) is at least partly attributable to this information loss, eliminating or reducing the resampling step is expected to yield a direct improvement in on-board accuracy.

Extension to multi-label classification. The current model treats arrhythmia classification as a single-label multi-class problem, assigning each heartbeat to exactly one of five categories. In clinical practice, however, a patient may present multiple coexisting conditions simultaneously — for example, bradycardia combined with a ventricular ectopic beat. Extending the system to multi-label classification would require replacing the final softmax layer with a set of independent sigmoid outputs, one per condition, each producing a probability that can be thresholded independently. The encoding pipeline and the convolutional backbone would remain unchanged; only the output layer and the loss function would need to be adapted. More broadly, the same architecture could be retrained on other cardiac pathologies or on entirely different arrhythmia taxonomies, since the image-encoding

approach is agnostic to the specific class definitions.

Real-world applications. Time-series data collected from sensors pervades virtually every engineering and medical domain: industrial machinery monitored for predictive maintenance, smart-grid meters tracking power consumption, environmental sensor networks, logistics fleets, smart buildings, and continuous patient monitoring are all domains where compact, low-power classifiers running at the edge would reduce latency, bandwidth, and privacy exposure compared to cloud-based alternatives. The system presented in this work sits squarely at that intersection. The most immediate deployment scenario is continuous cardiac monitoring on wearable devices: a board with a form factor comparable to the Nicla Vision or a smaller successor could be embedded in a smartwatch or a stick-on ECG patch, classifying each heartbeat in real time without any cloud connectivity. This is particularly relevant for patients with known arrhythmia risk who require long-term monitoring outside a clinical setting, where network access is unreliable and battery life is critical. A second scenario is hospital-grade embedded screening: a low-cost dedicated device could perform arrhythmia triage at the bedside without relying on external infrastructure, thereby reducing both costs and response time. Beyond cardiac monitoring, the same pipeline GAF encoding followed by a lightweight CNN compiled via Z-Ant is domain-agnostic: retraining on vibration signals from industrial machinery, power-consumption traces from smart meters, or motion data from logistics assets requires no architectural change, only a new labeled dataset and a recompilation of the library. A particularly compelling use case in the environmental domain is edge-based climate and air-quality monitoring: sensor nodes deployed in remote locations forests, mountain stations, or ocean buoys could classify anomalous temperature, humidity, or pollutant patterns locally, transmitting only a compact class label rather than a continuous raw stream, dramatically reducing the ì communication overhead on low-bandwidth links such as LoRa or satellite. In all these scenarios, a key advantage of the on-device approach is privacy: raw sensor data which may carry sensitive personal or operational information never leaves the device.

REFERENCES

[1] Mohammad Kachuee, Shayan Fazeli, and Majid Sarrafzadeh. ECG heartbeat categorization dataset. `https://www.kaggle.com/datasets/shayanfazeli/heartbeat`, 2018. Derived from the MIT-BIH Arrhythmia Database.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[3] George B. Moody and Roger G. Mark. The impact of the MIT-BIH arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3):45–50, 2001.

[4] STMicroelectronics. *STM32H747xI/G – Dual-Core Arm Cortex-M7 + Cortex-M4 MCU, Reference Manual RM0399*, 2023.

[5] Zhiguang Wang and Tim Oates. Imaging time-series to improve classification and imputation. *arXiv preprint arXiv:1506.00327*, 2015.

[6] Z-Ant Contributors. Z-Ant: An open-source embedded inference framework. `https://github.com/ZantFoundation/Z-Ant`, 2024–2025. Accessed: May 2026.